📄 Hybrid Parallel Graph Processing using MPI + OpenMP

Comprehensive System Design & Implementation Roadmap

---

1. Problem Formulation (Deep Level)

1.1 Computational Bottlenecks

Large-scale graph processing encounters three primary bottlenecks:

- **Memory Constraint**
  A graph with **1 billion nodes and 10 billion edges** cannot fit into a single machine's memory.
  Even with adjacency lists:
  - 8 bytes per edge → **~80 GB memory required**
- **Time Complexity**
  - BFS: **O(V + E)**
  - PageRank: **O(iterations × E)**
    On billion-scale graphs → **each iteration takes minutes sequentially**
- **Irregular Memory Access**
  Graphs involve pointer-chasing → unpredictable memory access
  → severe **CPU cache inefficiency** (major hidden bottleneck)

1.2 Why Sequential Approaches Fail

- Single-core execution → extremely slow (hours/days)
- No parallel processing of independent nodes
- Memory bandwidth saturated due to random access
- Limited by single-machine RAM capacity

1.3 Why Naive Parallelization is Insufficient

- OpenMP alone cannot scale → graph too large for one machine
- MPI alone underutilizes CPU cores within a machine
- Race conditions in shared memory updates
- Load imbalance due to high-degree nodes (power-law graphs)

1.4 Key Research Challenges

Challenge	Description
Load Balancing	Power-law graphs create uneven workload distribution
Communication Overhead	Cross-partition edges require frequent MPI messaging
Synchronization	Global barriers needed (BFS levels, PageRank iterations)
False Sharing	Cache line contention in OpenMP

---

2. System Architecture Design

2.1 Layered Architecture

```
DATA LAYER
→ Edge-list input → converted to CSR format

GRAPH PARTITIONING LAYER
→ 1D vertex partitioning
→ Each MPI rank owns vertex subset
→ Ghost vertices maintained

COMPUTATION LAYER
→ MPI rank executes local computation
→ OpenMP parallelizes within rank

COMMUNICATION LAYER
→ MPI_Allreduce → convergence
→ MPI_Sendrecv → boundary updates
→ MPI_Bcast → initialization
```

2.2 Hybrid Model Responsibilities

MPI Handles:

- Graph distribution across nodes
- Communication of ghost vertices
- Global synchronization

OpenMP Handles:

- Parallel edge traversal
- Multi-threading local computations
- Efficient CPU utilization

2.3 Process–Thread Mapping

- 1 MPI rank per NUMA node
- OpenMP threads = number of cores per node

---

3. Graph Partitioning Strategy

3.1 Comparison

Method	Description	Pros	Cons
Edge-cut (1D)	Split vertices	Simple	High communication
Vertex-cut (2D)	Split edges	Better balance	Complex
2D Block	Matrix partition	Scalable	Very complex

3.2 Recommended Strategy

1D Vertex Partitioning (Edge-cut)

Reasons:

- Simpler implementation
- Suitable for BFS, SSSP, PageRank
- Fits project constraints

3.3 Optimization Insight

- Sort vertices by degree
- Interleave across partitions
  → reduces load imbalance from hub nodes

---

4. Execution Flow Pipeline

Step-by-Step Process

1. **Data Loading**
   - Rank 0 reads edge list
   - Converts to CSR format
2. **Graph Distribution**
   - MPI_Scatterv distributes partitions
   - Ghost vertices identified
3. **Initialization**
   - BFS/SSSP → source broadcast
   - PageRank → uniform initialization
4. **Local Computation**
   - OpenMP parallel loops

    - Thread-local updates
5. **Communication**
    - Exchange ghost vertex data via MPI
6. **Synchronization**
    - MPI_Barrier ensures consistency
7. **Convergence Check**
    - MPI_Allreduce for global condition
8. **Result Gathering**
    - MPI_Gather to Rank 0

---

5. Algorithm Adaptation

5.1 BFS (Breadth First Search)

- Distributed frontier processing
- MPI communication for cross-partition vertices
- Level-synchronous execution

5.2 SSSP (Bellman-Ford)

- Parallel edge relaxation
- Ghost vertex updates via MPI
- Convergence via global reduction

5.3 PageRank

- Iterative rank updates
- Cross-partition data exchange
- Convergence using delta threshold

---

6. Dataset Selection

Dataset	Nodes	Edges	Type	Use
Karate Club	34	78	Social	Debugging
Email-EuAll	265K	420K	Email	Testing
roadNet-CA	1.97M	5.5M	Road	Medium
com-LiveJournal	3.9M	34.7M	Social	Benchmark
com-Friendster	65.6M	1.8B	Social	Stress test

Sources

- SNAP (Stanford)
- KONECT
- LAW datasets

---

7. Performance Evaluation Plan

Metrics

- Execution Time
- Speedup: $S = T_{seq}/T_{parallel}$
- Efficiency: $E = S/P$
- Strong Scaling
- Weak Scaling

Baselines

Configuration	Description
Sequential	Single core
OpenMP	Multi-thread
MPI	Multi-process
Hybrid	MPI + OpenMP

Expected Outcome

Config	Speedup
Sequential	1x
OpenMP	~5–6x
MPI	~3–4x
Hybrid	~14x

---

8. Key Challenges & Mitigation

Challenge	Solution
Communication Overhead	Non-blocking MPI
Load Imbalance	Degree-based partitioning
False Sharing	Cache-line padding
Network Latency	Message batching

Challenge	Solution
Deadlocks	MPI_Sendrecv / async
Race Conditions	Atomic operations

---

9. Research Extensions

- Dynamic load balancing
- Asynchronous MPI
- Delta-stepping SSSP
- Graph compression
- GPU integration (CUDA)

---

10. Implementation Roadmap

Phase	Duration	Tasks
Setup	Days 1–3	Install MPI, datasets
Sequential	Days 4–7	Baseline algorithms
OpenMP	Days 8–11	Thread parallelism
MPI	Days 12–17	Distributed system
Hybrid	Days 18–22	Combine both
Evaluation	Days 23–28	Benchmark & report

---

11. Final Deliverables

Report Structure

- Abstract
- Introduction
- Architecture
- Algorithm Design
- Implementation
- Experiments
- Results
- Challenges
- Conclusion

Graphs to Include

- Speedup vs processes
- Speedup vs threads

- Execution comparison
- Communication vs computation
- Weak scaling